```
-- tcp-server.hs
import Network
import System.IO

main :: IO ()
main = withSocketsDo $ do
         sock <- listenOn $ PortNumber 3001
         putStrLn "Starting server ..."
         handleConnections sock

handleConnections :: Socket -> IO ()
handleConnections sock = do
  (handle, host, port) <- accept sock
  output <- hGetLine handle
  putStrLn output
  handleConnections sock
```

The main function starts a server on port 3001 and transfers the socket handler to a *handleConnections* function. It accepts any connection requests, reads the data, prints it to the server log, and waits for more clients.

First, you need to compile the *tcp-server.hs* and *tcp-client.hs* files using GHC:

```
$ ghc --make tcp-server.hs
[1 of 1] Compiling Main    ( tcp-server.hs, tcp-server.o )
Linking tcp-server ...

$ ghc --make tcp-client.hs
[1 of 1] Compiling Main    ( tcp-client.hs, tcp-client.o )
Linking tcp-client ...
```

You can now start the TCP server in a terminal:

```
$ ./tcp-server
Starting server ...
```

Then run the TCP client in another terminal:

```
./udp-client
```

You will now observe the 'Hello, world' message printed in the terminal where the server is running:

```
$ ./tcp-server
Starting server ...
Hello, world!
```

The *Network.Socket* package exposes more low-level socket functionality for Haskell and can be used if you need finer access and control. For example, consider the following UDP (user datagram protocol) client code:

```
-- udp-client.hs
import Network.Socket

main :: IO ()
main = withSocketsDo $ do
         (server:_) <- getAddrInfo Nothing (Just "localhost")
(Just "3000")
         s <- socket (addrFamily server) Datagram
defaultProtocol
         connect s (addrAddress server)
         send s "Hello, world!"
         sClose s
```

The *getAddrInfo* function resolves a host or service name to a network address. A UDP client connection is then requested for the server address, a message is sent, and the connection is closed. The type signatures of *getAddrInfo*, *addrFamily* and *addrAddress* are given below:

```
ghci> :t getAddrInfo
getAddrInfo
  :: Maybe AddrInfo
     -> Maybe HostName -> Maybe ServiceName -> IO [AddrInfo]

ghci> :t addrFamily
addrFamily :: AddrInfo -> Family

ghci> :t addrAddress
addrAddress :: AddrInfo -> SockAddr
```

The corresponding UDP server code is as follows:

```
-- udp-server.hs

import Network.Socket

main :: IO ()
main = withSocketsDo $ do
         (server:_) <- getAddrInfo Nothing (Just "localhost")
(Just "3000")
         s <- socket (addrFamily server) Datagram
defaultProtocol
         bindSocket s (addrAddress server) >> return s
         putStrLn "Server started ..."
         handleConnections s

handleConnections :: Socket -> IO ()
handleConnections conn = do
  (text, _, _) <- recvFrom conn 1024
  putStrLn text
  handleConnections conn
```

The UDP server binds to localhost and starts to listen on Port 3000. When a client connects, it reads a maximum of 1024 bytes of data, prints it to *stdout*, and waits to accept